Data Engineering Take-Home Assignment

Overview

At YipitData, we work with many different types of data. Recently, we have been asked to build an intelligent ETL pipeline that processes technology news articles, enriches them using AI/ML embeddings, and enables semantic search using vector similarity. This assignment tests your ability to integrate AI tools with modern data engineering practices.

Dataset Description

You will receive two files:

- `tech_news.csv` - Contains news articles about technology companies (500 rows)
- `company_metadata.json` - Contains additional company information

The data is intentionally messy and requires cleaning and transformation.

Your Task

Part 1: Core ETL Pipeline

Create essential data cleaning functions:

1. **Revenue Cleaning** - Clean the "revenue" column:

- Convert all amounts to USD (handle EUR, GBP, JPY)
- Handle ranges (e.g., "$10M - $20M") by taking the midpoint
- Parse formats: "5.2B", "$5,200,000,000", "5.2 billion"
- Convert NaN/null/N/A/"Not disclosed" to 0
- Return as integer

Currency Conversion (Approximate rates for this exercise):

EUR to USD: multiply by 1.1
GBP to USD: multiply by 1.27
JPY to USD: divide by 150

2. **Date Normalization** - Clean the "published_date" column:

- Handle multiple date formats (ISO, US, EU formats)
- Extract year, quarter, month for analysis
- Return as datetime object
- Handle edge cases (invalid dates, missing values)

3. **Category Standardization** - Normalize the "category" column:

- Map similar categories (e.g., "AI/ML", "Artificial Intelligence", "Machine Learning" → "AI_ML")
- Create a consistent taxonomy (provide mapping in your code)

4. Company Metadata Integration & Validation

a. Company Name Validation
   i. Validate the company names in articles exist in the metadata
   ii. Log or flag articles with companies not found in metadata
   iii. Consider fuzzy matching for slight variations (e.g., "Amazon Web Services" vs "AWS")

b. Metadata Enrichment
   i. Join company metadata to articles by company name
   ii. Add derived fields: `company_age`: Calculate age based on `founded_year` and article `published_date`, `company_size_category`:
   iii. Categorize by employee_count (e.g., "Small" < 10K, "Medium" 10K-30K, "Large" > 30K)
   iv. Include all metadata fields in the final output

c. Industry-Based Filtering
   i. Use the `industry` field from metadata as an additional filter option
   ii. The metadata contains industries like "AI/ML", "Data Analytics", "Cloud Computing", etc.
   iii. Consider both article `category` AND company `industry` when filteringJoin with the company metadata section

Part 2: Integration with Vector Database

Implement semantic search using embeddings:

Requirements

1. **Text Embedding Generation**

- Generate embeddings for article titles + summaries (concatenated)
- Use `sentence-transformers` library with `all-MiniLM-L6-v2` model or equivalent
- Store embeddings as numpy arrays

2. **Vector Similarity Search**

- Implement cosine similarity search
- Function: `find_similar_articles(query_text, top_k=5)`
- Return article IDs and similarity scores

3. **DuckDB Integration with Vector Storage**

- Load cleaned data into DuckDB
- Store embeddings using DuckDB's ARRAY type or as JSON
- Create a function for hybrid search: SQL filters + vector similarity
- Example: "Find AI-related articles from 2022-2024 with revenue > $50M, similar to article X"

Part 3: Query Interface & Export

Required Deliverable:

Create a pipeline that exports a **CSV file** containing:

- All articles about "AI" or "Machine Learning" companies (filter by category and or industry)
- Published between 2022-2024
- With revenue >= $50M USD
- Include columns: `article_id`, `title`, `company_name`, `published_date`, `category`, `revenue_usd`, `summary`, `url`, `industry`, `founded_year`, `headquarters`, `employee_count`, `is_public`, `stock_ticker`, `company_age`, `company_size_category` , `embedding` (as array/list), top_similar_articles,
- Add a `top_similar_articles` column containing IDs of top 3 most similar articles

Query Functions:

1. Function to execute SQL queries on DuckDB
2. Function to perform vector similarity search
3. Export function to Csv with embeddings

Deliverable Requirements

1. Code Implementation

- Clean, modular Python code organized in functions/classes
- A main pipeline script that processes the data and generates the output
- Proper error handling for edge cases

2. Documentation (Keep it concise!)

README.md (Max 1 page):

- Installation instructions (dependencies)
- How to run the pipeline (step-by-step)
- Example usage of key functions
- System requirements

EXPLAIN.md (Max 1.5 pages):

- Your approach and key decisions
- Choice of embedding model and why
- How you handled edge cases in data cleaning
- Trade-offs you made (performance vs. accuracy)
- What you would improve with more time

requirements.txt:

- All dependencies with versions

3. Output Files

- `ai_articles_enriched.csv` - The filtered and enriched dataset

Evaluation Criteria

Technical Excellence (45%)

- Code quality and organization
- Correct implementation of vector search
- Efficient data processing

AI/ML Integration (25%)

- Proper use of embedding models
- Quality of semantic search results
- Effective vector similarity implementation

Data Engineering Best Practices (20%)

- Data validation and quality checks
- Proper use of DuckDB features
- Clean data transformations

Documentation (10%)

- Clear README with working instructions
- Well-explained decisions in EXPLAIN.md

Submission

Please submit the ZIP file to **LINK** containing:

- All source code (organized in a clear structure)
- README.md
- EXPLAIN.md
- requirements.txt

- The output CSV file (`ai_articles_enriched.csv`)

Important:

- Ensure we can run your code by following the README
- Test your installation instructions
- Focus on working code over perfect code

Questions?

Document your assumptions in EXPLAIN.md and proceed with your best judgment. This reflects real-world scenarios where you need to make decisions with incomplete information.

Good luck! We're excited to see your approach to modern data engineering pipelines.

Hints & Quick Start Guide

Recommended Libraries

```
pip install pandas duckdb numpy scikit-learn sentence-transformers pyarrow
```

Quick Start Code Snippets

1. Loading Sentence Transformers:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(texts)  # texts is a list of strings
```

2. Cosine Similarity:

```
from sklearn.metrics.pairwise import cosine_similarity
similarity_scores = cosine_similarity([query_embedding], all_embeddings)[0]
```

3. DuckDB with Arrays:

```
import duckdb
conn = duckdb.connect(':memory:')
# Store embeddings as JSON or use ARRAY type
df['embedding_json'] = df['embedding'].apply(lambda x: x.tolist())
conn.execute("CREATE TABLE articles AS SELECT * FROM df")
```